클라우드 컨테이너 실습 과제

나만의 컨테이너 만들기

Developer Sandbox(OpenShift)에 나만의 컨테이너 서비스를 배포하고 Git으로 제출하기

항목	내용
문의 사항	kako@redhat.com
과목명	지능형IoT플랫폼(Intelligent IoT Platform)
제출 기한	【2026-10-30 10:00】
제출물	① 컨테이너 소개 문서(README.md) ② 배포에 성공한 YAML 파일 → 본인 **Git 저장소에 Push 후 저장소 URL**을 **8**주차 과제로 제출하시면 됩니다.
기준 저장소	https://github.com/{본인계정}/{과제_저장소}
채점 방식	제출된 Git의 YAML로 채점자가 직접 배포하여 성공 여부 확인

목차

1. 과제 개요

2. 컨테이너 주제 정하기

3. 제출 저장소 구조

4. 수행 절차

5. Developer Sandbox 주의사항 및 자주 발생하는 오류

6. 채점 기준

7. AI 및 검색 활용 안내

8. 제출 전 체크리스트

1. 과제 개요

1.1 과제 목적

지난 두 번의 수업에서 **Podman**과 **Developer Sandbox(OpenShift** 기반**)**로 기본 컨테이너를 배포하고 실행해 보았습니다. 이번 과제에서는 **어떤 컨테이너를 어떤 구성으로 띄울지 스스로 정하고**, 본인의 Developer Sandbox 계정에서 직접 배포해 동작을 확인한 뒤, **성공한 결과물(YAML)과 소개 문서를 Git에 Push**하여 제출합니다.

이 과제에서 "나만의 컨테이너"란 이미지를 직접 빌드하는 것이 아니라, **주제를 정하고 그에 맞게 이미지 선택·설정·Object 구성을 설계하여 하나의 서비스로 완성하는 것**을 말합니다. 별도의 Podman 빌드는 필요하지 않습니다.

과제를 마치면 다음을 스스로 할 수 있어야 합니다.

- 목적에 맞는 컨테이너 이미지를 고르고, OpenShift에서 동작하도록 설정하기
- Deployment, Service, Route 등 Object를 YAML로 작성하여 Developer Sandbox에 배포하기
- 배포 결과를 확인하고, 다른 사람이 Git만 보고 똑같이 배포할 수 있도록 정리하기
- 내가 만든 컨테이너 서비스가 무엇이고 어떻게 구성되어 있는지 문서로 설명하기

1.2 수행 요구사항 요약

No	요구사항	세부 내용
1	컨테이너 소개 문서	어떤 컨테이너(서비스)인지, 무엇을 하는지, 아키텍처 구성과 사용한 Object를 README.md에 작성
2	Git 저장소 생성	과제 제출을 위한 Git 저장소를 생성하고, 본인 학번 폴더를 만들어 그 안에 결과물을 추가
3	Sandbox 배포 및 Push	본인 Developer Sandbox에 Deployment·Service·Route를 배포해 성공을 확인하고, 사용한 YAML을 Git에 Commit·Push
4	AI·검색 활용	AI 도구와 검색 활용 가능. 사용 내역을 README에 간단히 기록 (7장 참고)
5	추가 Object (선택)	ConfigMap, Secret, PVC, Probe 등 수업에서 배운 Object를 자유롭게 추가 활용 (가산점)

2. 컨테이너 주제 정하기

공개된 컨테이너 이미지를 활용하되, **본인만의 목적과 설정**이 담긴 서비스를 만듭니다. 채점자가 Route 주소로 접속해 동작을 확인하므로 **웹 브라우저나 curl로 응답을 확인할 수 있는 서비스**를 권장합니다.

난이도	예시 주제	활용하면 좋은 **Object**
기본	나만의 자기소개 웹페이지 (웹서버 이미지 + ConfigMap으로 HTML 주입)	Deployment, Service, Route, ConfigMap
중급	환경변수로 문구·설정이 바뀌는 웹앱 관리자 비밀번호가 있는 웹 도구	+ Secret, Liveness/Readiness Probe
심화	웹 + DB(Redis·PostgreSQL 등) 2-tier 서비스 데이터가 유지되는 메모·방명록 서비스	+ PVC, 다중 Deployment

이미지 선택 팁

OpenShift는 보안상 컨테이너를 **root가 아닌 임의의 사용자**로 실행합니다. **Red Hat UBI 이미지**(`registry.access.redhat.com/ubi9/...`)나 이름에 `unprivileged`가 붙은 이미지처럼 비루트 실행을 지원하는 이미지를 고르세요.

Docker Hub 이미지는 다운로드 횟수 제한에 걸릴 수 있으므로 `registry.access.redhat.com`, `quay.io` 이미지를 우선 사용하세요.

Sandbox는 리소스가 제한된 공유 환경입니다. 무거운 서비스(대형 DB, AI 모델 등)는 피하세요.

3. 제출 저장소 구조

본인 학번 폴더 안에 작업물을 저장합니다. 채점자는 이 폴더의 `k8s/` 디렉터리를 그대로 배포합니다.

```
<저장소 루트>/
└── students/
    └── 2026XXXXXX/          ← 학번 폴더
        ├── README.md            ← 컨테이너 소개 · 아키텍처 · 배포 결과
        ├── k8s/                 ← 배포에 성공한 YAML (채점 대상)
        │   ├── deployment.yaml
        │   ├── service.yaml
        │   ├── route.yaml
        │   └── (configmap.yaml, secret.yaml, pvc.yaml ...)
        └── images/              ← 아키텍처 그림, 배포 성공 화면 캡처
```

3.1 README.md 필수 항목

섹션	작성 내용
1. 컨테이너 소개	서비스 이름, 한 줄 설명, 이 주제를 고른 이유, 주요 기능
2. 아키텍처	구성도(그림 또는 Mermaid), 사용자 → Route → Service → Pod 흐름, 사용한 Object 목록과 각 역할
3. 사용 이미지	이미지 주소와 태그, 선택한 이유, 노출 포트
4. 배포 방법	`oc apply -f k8s/` 명령, 배포 후 확인 방법(접속 경로와 예상 화면/응답)
5. 배포 결과	`oc get all` 출력 또는 Topology 화면, 브라우저 접속 화면 캡처
6. 어려웠던 점	겪은 오류와 해결 과정 (선택이지만 권장)
7. AI·참고자료 활용 내역	사용한 도구와 용도, 참고한 문서 링크

4. 수행 절차

Step 1. Git 저장소 디렉터리 생성

GitHub에서 생성한 본인 저장소에 `students/<학번>/` 폴더를 만듭니다. Git 명령이 익숙하지 않다면 GitHub 웹 화면의 **Add file → Create new file / Upload files** 기능을 사용해도 됩니다.

```
# Git 명령을 쓰는 경우
git clone https://github.com/<내계정>/<저장소명>.git
cd <저장소명>
mkdir -p students/2026XXXXXX/{k8s,images}
```

Step 2. Developer Sandbox에 배포 및 확인

본인 Developer Sandbox 계정에서 컨테이너를 배포합니다. 배포 방법은 수업에서 배운 방식 중 편한 것을 사용하면 됩니다.

방법	진행 방법
A. 웹 콘솔	**+Add** 메뉴(Container images, Import YAML 등)로 배포
B. oc 명령	웹 콘솔 우측 상단 계정 메뉴의 **Copy login command**로 로그인한 뒤 `oc apply -f k8s/` 실행

배포 후 다음을 확인합니다.

- Pod 상태가 **Running**이고, Route 주소(`https://<host>`)로 접속했을 때 의도한 화면/응답이 나오는지 확인합니다.
- `oc get all` 출력(또는 웹 콘솔 Topology 화면)과 브라우저 접속 화면을 캡쳐해 `images/`에 저장하고 README에 넣습니다.
- 배포에 사용한 YAML을 `k8s/` 폴더에 저장합니다. 웹 콘솔에서 복사한 YAML이라면 `status`, `namespace`, `uid`, `resourceVersion`, `creationTimestamp`, `managedFields`, Route의 `host` 같은 자동 생성 항목은 지웁니다.
- **재현성 검증:** 리소스를 모두 삭제(`oc delete -f k8s/`)한 뒤 `k8s/` 폴더의 YAML만으로 다시 배포해 정상 동작하는지 확인합니다. 채점도 이 방식으로 진행됩니다.

Step 3. Commit · Push · 제출

```
git add students/2026XXXXXX
git commit -m "2026XXXXXX 홍길동 - 자기소개 웹 컨테이너"
git push origin main
```

Git 저장소의 URL을 제출합니다. 제출 후에는 기한까지 같은 브랜치에 계속 Push해도 되며, **기한 시점의 마지막 Commit**으로 채점합니다.

5. Developer Sandbox 주의사항 및 자주 발생하는 오류

Developer Sandbox는 여러 사용자가 함께 쓰는 공유 OpenShift 클러스터로, 사용자마다 프로젝트(namespace) 하나와 CPU·메모리·스토리지 할당량이 제한됩니다. **Developer Sandbox 무료 사용 기간(30일)이 지나면 환경을 재신청하여 새로운 환경을 제공 받으시면 됩니다.** 오래 실행된 Pod는 자동으로 축소(idle)될 수 있으므로, 접속이 안 되면 Pod 상태부터 확인하세요.

증상	원인	해결 방법
ImagePullBackOff	이미지 주소·태그 오타, 비공개 이미지	이미지 주소 재확인, 공개 이미지 사용
toomanyrequests	Docker Hub 익명 다운로드 횟수 제한	registry.access.redhat.com 또는 quay.io 이미지 사용
CrashLoopBackOff Permission denied	OpenShift는 비루트 임의 UID로 실행 → root 전제 이미지는 실패	UBI 이미지 또는 unprivileged 이미지 사용, `oc logs <pod>`로 원인 확인
bind: permission denied (포트 80)	비루트 사용자는 1024 미만 포트 사용 불가	8080 등 1024 이상 포트를 쓰는 이미지·설정 사용
Route 접속 시 Application is not available	Service selector와 Pod label 불일치, 포트 불일치, Pod 미기동	`oc get endpoints`로 연결 확인, label·port 일치시키기
Pending / exceeded quota	할당량 초과 또는 resources 미지정	불필요한 리소스 삭제, requests/limits를 작게 지정
apply 시 오류 (field is immutable 등)	콘솔에서 복사한 YAML에 uid·clusterIP 등 자동 생성 필드가 남아 있음	Step 2에서 안내한 자동 생성 항목 제거 후 다시 적용

6. 채점 기준

평가 항목	세부 기준	배점
배포 성공 (핵심)	k8s/ 폴더만으로 오류 없이 배포, Pod Running, Route 접속 시 정상 응답	40
컨테이너 소개 · 아키텍처 문서	주제 소개의 명확성, 구성도와 Object 설명의 정확성, 배포 결과 증빙	25
YAML 품질	Deployment·Service·Route 정상 연결, 이름 규칙 준수, namespace·자동생성 필드 없음, resources 지정	20
추가 Object 활용	ConfigMap, Secret, PVC, Probe 등을 주제에 맞게 활용하고 README에 설명	15
합계		100

감점 및 0점 처리 기준

저장소에 본인 폴더가 없거나, 다른 학생의 폴더를 수정한 경우

YAML로 배포가 되지 않는 경우 → 배포 성공 항목 0점

타인의 결과물을 그대로 제출한 경우 0점

7. AI 및 검색 활용 안내

ChatGPT, Claude, Copilot 등 AI 도구와 웹 검색을 자유롭게 활용할 수 있습니다. 다만 아래 사항을 지켜 주세요.

- README의 "AI·참고자료 활용 내역"에 사용한 도구와 용도를 간단히 적습니다. (예: "Deployment YAML 초안 작성에 Claude 활용, 권한 오류는 Red Hat 문서 참고")
- AI가 만든 YAML은 반드시 본인 **Sandbox에 직접 배포해서 동작을 확인**한 뒤 제출합니다. AI 답변에는 Docker Hub 이미지, root 실행, 포트 80처럼 OpenShift에서 바로 동작하지 않는 설정이 자주 포함됩니다.
- 제출한 모든 YAML의 각 항목이 무슨 역할인지 본인이 설명할 수 있어야 합니다.

8. 제출 전 체크리스트

- ☐ 본인 학번 폴더 안에만 파일을 추가
- ☐ README에 컨테이너 소개 · 아키텍처 · 사용 이미지 · 배포 방법 · 결과 캡처 · AI 활용 내역을 작성
- ☐ YAML에 namespace, Route host, uid·status 등 자동 생성 필드가 없음
- ☐ `oc delete -f k8s/` 후 `oc apply -f k8s/`만으로 다시 정상 배포됨을 확인
- ☐ Route 주소로 접속해 정상 응답을 확인
- ☐ 변경 사항을 Push하고 저장소 URL을 제출